

OPTIMAL DECISION MAKING UNDER UNCERTAINTY

강화학습

Reinforcement Learning

정답을 알려주는 사람이 없어도,
시행착오만으로 스스로 똑똑해지는 법.

윤서진

AI Researcher · HD한국조선해양 (KSOE)



01

SPEAKER

윤서진.

Seojin Yoon

NOW

AI Researcher

HD한국조선해양 (KSOE)

SCHOOL

서강대학교 AI·SW대학원

67기

LINK

linkedin.com/in/tildalice

강화학습 · 시행착오를 통한 지능의 진화

WHY THIS TALK

제가 이 이야기를 하는 이유

배를 만드는 현장에서 AI를 다루다 보면, 결국 “정답이 없는 상황에서 계속 결정을 내려야 하는 문제”와 마주칩니다.

강화학습은 그 문제를 다루는 언어입니다. 오늘은 그 언어의 기본 문법만 챙겨 가시도록 준비했습니다.

TODAY

수식을 외우는 시간이 아니라,
관점 하나를 가져가는 시간입니다.

PROLOGUE

아무도 정답을 알려주지 않았습니니다.

그런데 이 아이는 **걷는 법**을 배웁니다.
넘어지고, 아파하고, 다시 일어나면서.

— 오늘 우리가 이야기할 학습 방식이 정확히 이것입니다.



WEBINAR ROADMAP

오늘의 여정

개념 → 수학 → 알고리즘 → 딥러닝 →
최신 응용.
다섯 계단을 차례로 올라갑니다.

강화학습의 개념과 철학

01

문제를 수학으로 정의하기 — MDP

02

고전적 강화학습 — Q-Learning

03

딥러닝과의 만남 — DQN

04

강화학습의 확장 — Physical AI · LLM

05

TAKE - AWAY

다섯 섹션을 관통하는 질문은 하나입니다.
“불확실한 세상에서, 어떻게 더 나은 선택을 배울까?”

SECTION

01

강화학습의 개념과 철학

기계에게 **정답** 대신 **보상**을 주면
어떤 일이 벌어질까요?

우리는 어떻게 배우는가

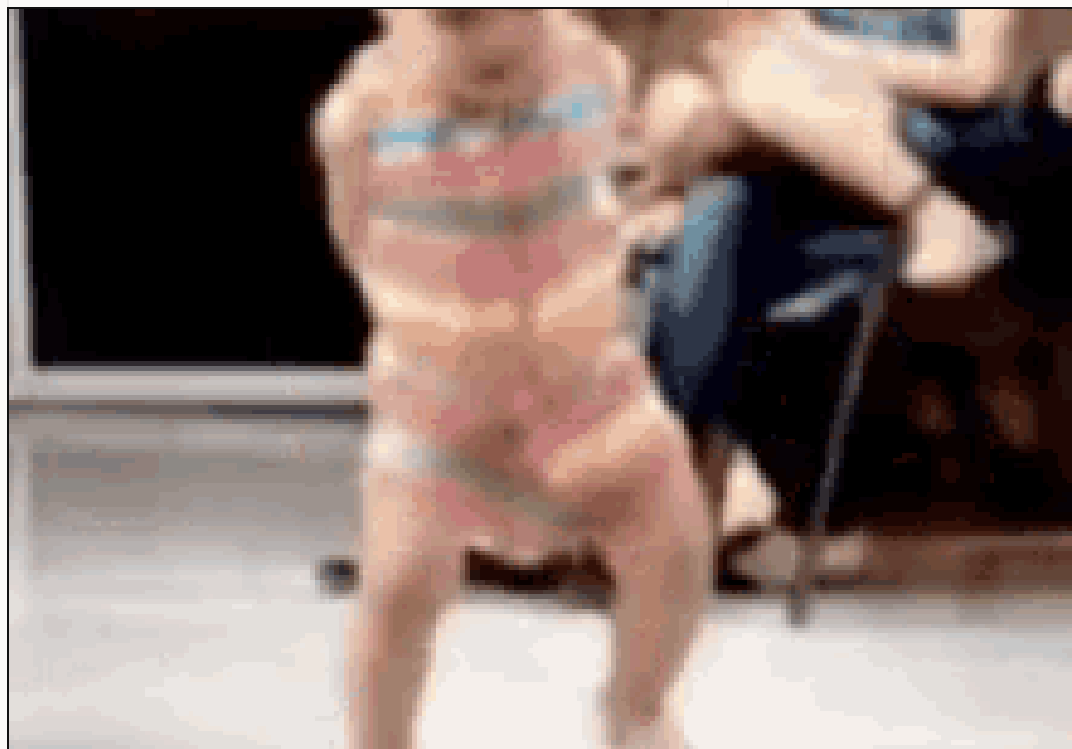
머신러닝의 세 갈래

에이전트-환경 루프

성공 사례

오프닝

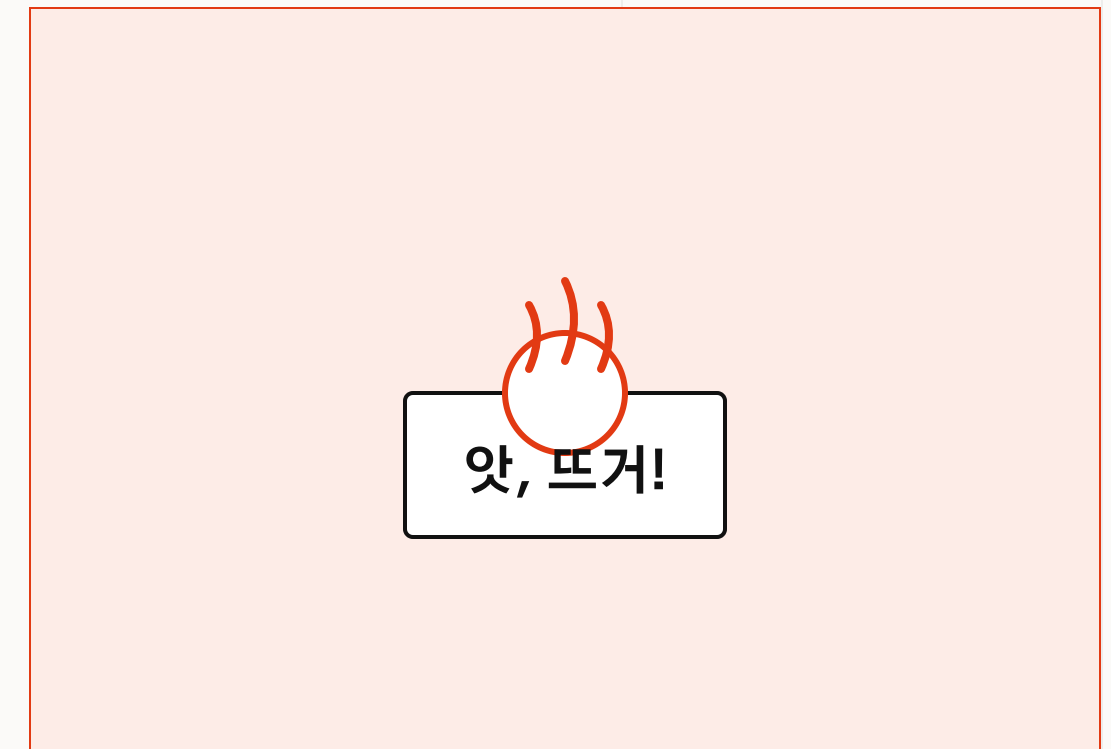
우리는 어떻게 배우는가



넘어짐 - 아픔이라는 음의 보상



간식 - 성공이라는 양의 보상



단 한 번의 경험 - 다시는 만지지 않음

MESSAGE 누가 정답을 알려주지 않아도, **행동의 결과**가 우리를 가르칩니다.

지도 · 비지도 · 강화

기계를 가르치는 3가지 방법

지도학습 Supervised Learning “정답이 뭐야?” 사람이 매긴 정답(label)을 보고 흉내 냅니다. 이미지 분류, 번역, 음성 인식. 필요한 것 — 정답이 달린 데이터 수만 장	비지도학습 Unsupervised Learning “어떻게 생겼어?” 정답 없이 데이터의 숨은 구조를 찾습니다. 군집화, 차원 축소, 이상 탐지. 필요한 것 — 그냥 데이터 더미	강화학습 Reinforcement Learning “뭘 해야 이득이야?” 정답 대신 보상(reward)을 받고, 그 점수를 높이는 행동을 찾습니다. 필요한 것 — 시도해 볼 수 있는 환경
--	---	--

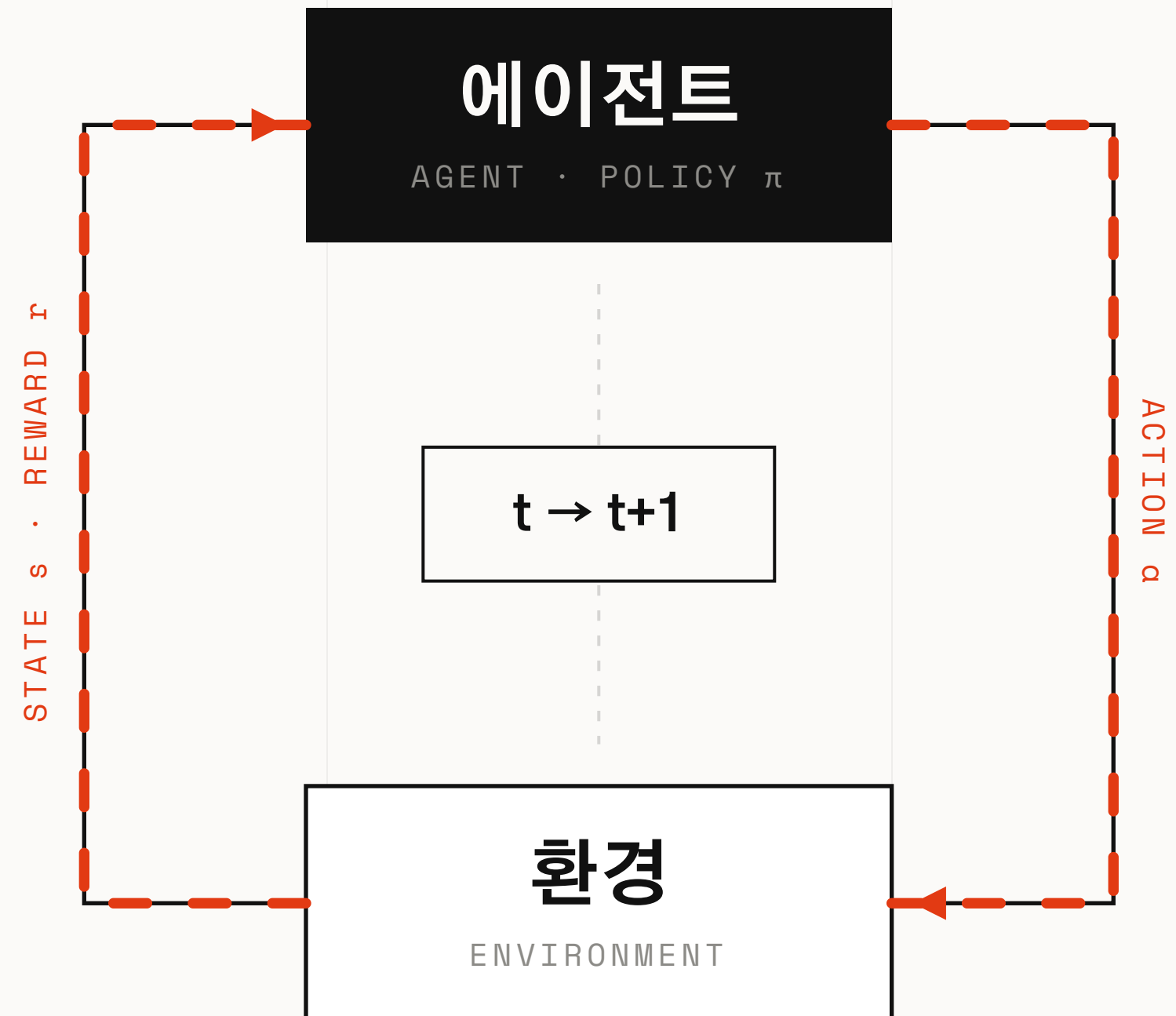
—— 앞의 둘은 데이터를 설명하고, 강화학습은 행동을 결정합니다. 목적 자체가 다릅니다.

강화학습의 심장

단 하나의 루프

강화학습에 등장하는 모든 알고리즘은
결국 이 루프를 **더 잘 도는 법**에 대한 이야기입니다.

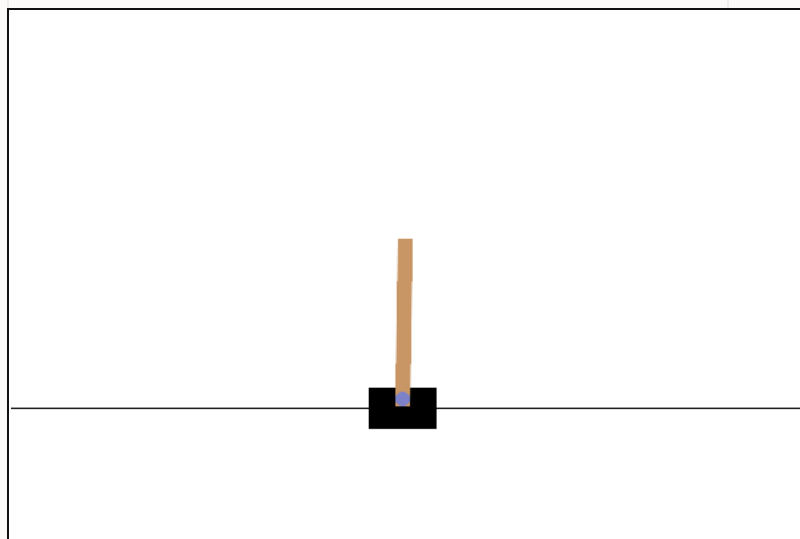
s_t	상태 — 지금 내가 처한 상황
a_t	행동 — 내가 고른 선택
r_t	보상 — 그 선택의 결과 점수
s_{t+1}	다음 상태 — 바뀐 상황, 그리고 반복



상태를 보고 → 행동하고 → 보상과 다음 상태를 받는다. 이 루프가 강화학습의 전부입니다.

백문이 불여일견

강화학습은 이런 데서 배웁니다



CartPole — 강화학습의 Hello World

행동 카트를 좌 / 우로

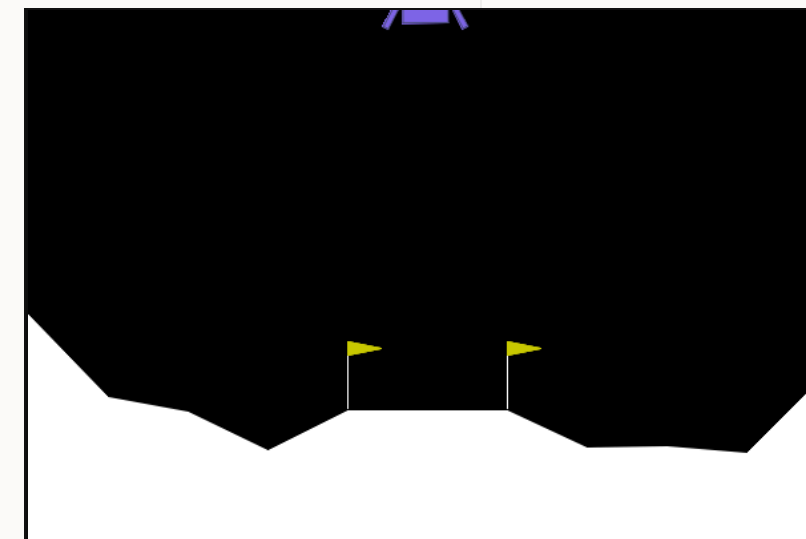
보상 버틴 1스텝마다 +1



FrozenLake — 우리가 본 그리드월드

행동 상 / 하 / 좌 / 우

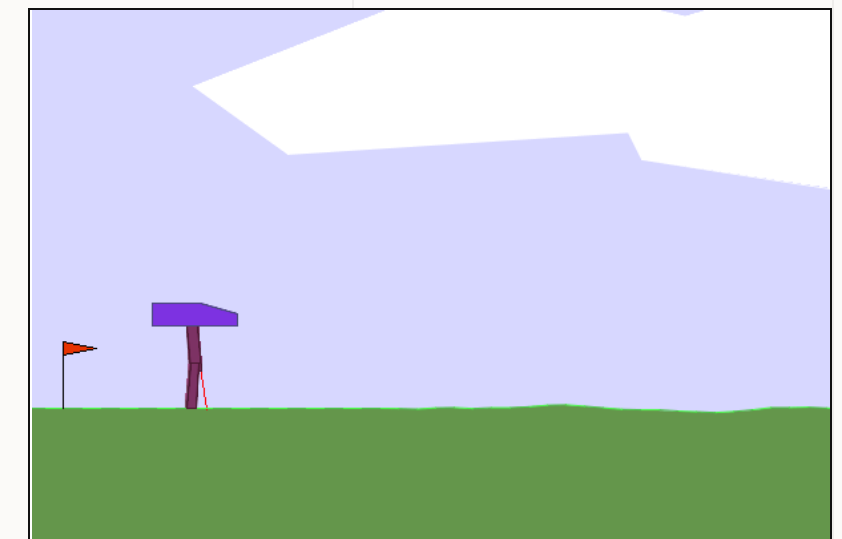
보상 선물 도착 +1, 구멍 0



LunarLander — 부드럽게 착륙시키기

행동 엔진 3개 점화

보상 곱게 착륙할수록 높게



BipedalWalker — 걷는 법 배우기

행동 관절 토크 — 연속값

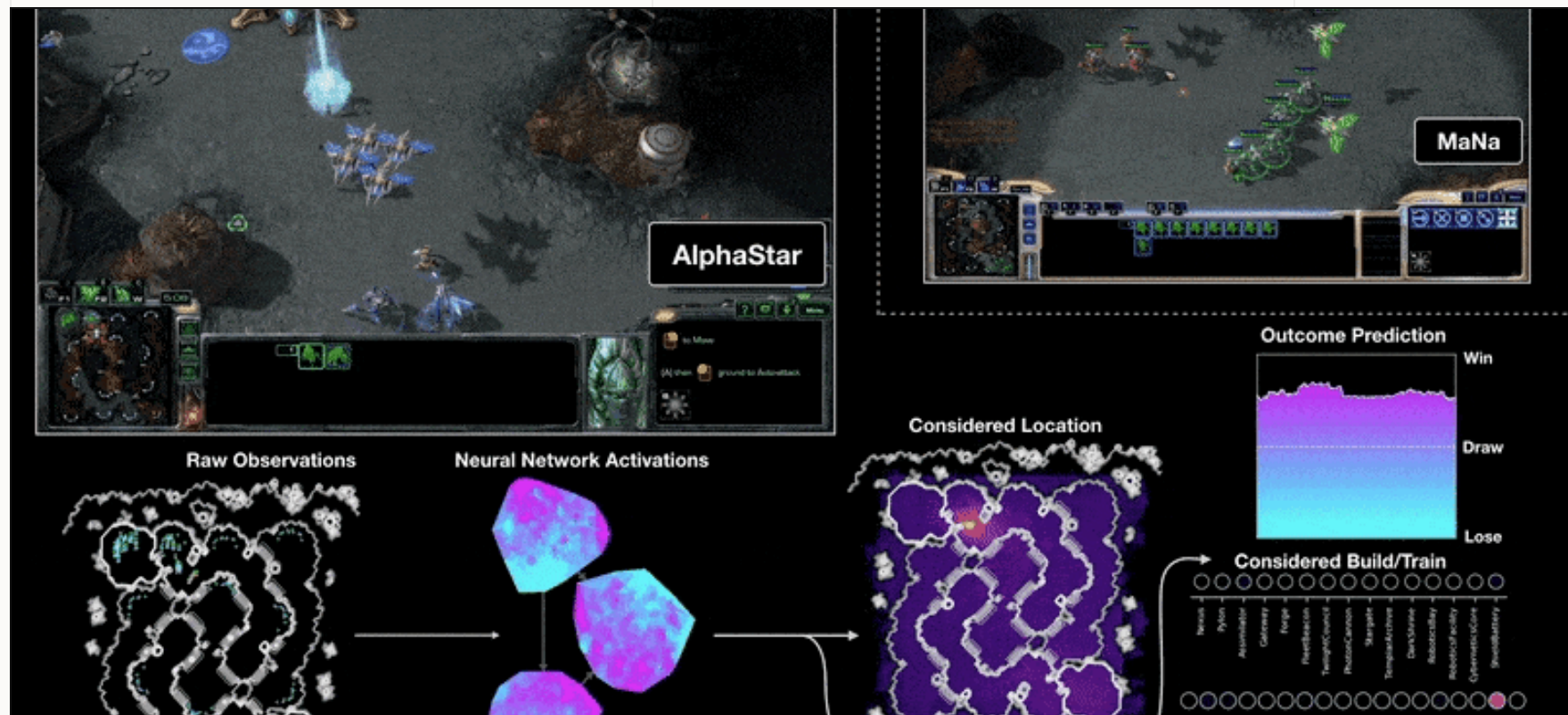
보상 앞으로 간 거리

NOTE

넷 다 정답표 없이 시행착오만으로 저 동작을 찾아냈습니다. 앞의 아기와 정확히 같은 방식으로요.

그래서 뭘 했는데?

강화학습이 실제로 해낸 일



AlphaStar - 스타크래프트 II 그랜드마스터 등급 도달 (DeepMind, 2019)

게임

Atari 벽돌깨기, **AlphaGo**(바둑), StarCraft II. 사람이 만든 최고 수준을 넘어선 첫 무대.

로보틱스

보행(Locomotion), 물체 조작(Manipulation). 넘어져도 다시 일어나는 법을 **시뮬레이션에서 수만 번** 연습합니다.

산업 · 인프라

구글 데이터센터 냉각 제어(**냉각 에너지 약 40% 절감**), 반도체 칩 배치 설계.

SECTION

02

문제를 수학으로 정의하기

Markov Decision Process

막연한 “학습”을 **풀 수 있는 문제로**
바꾸는 틀이 필요합니다.

순차적 의사결정

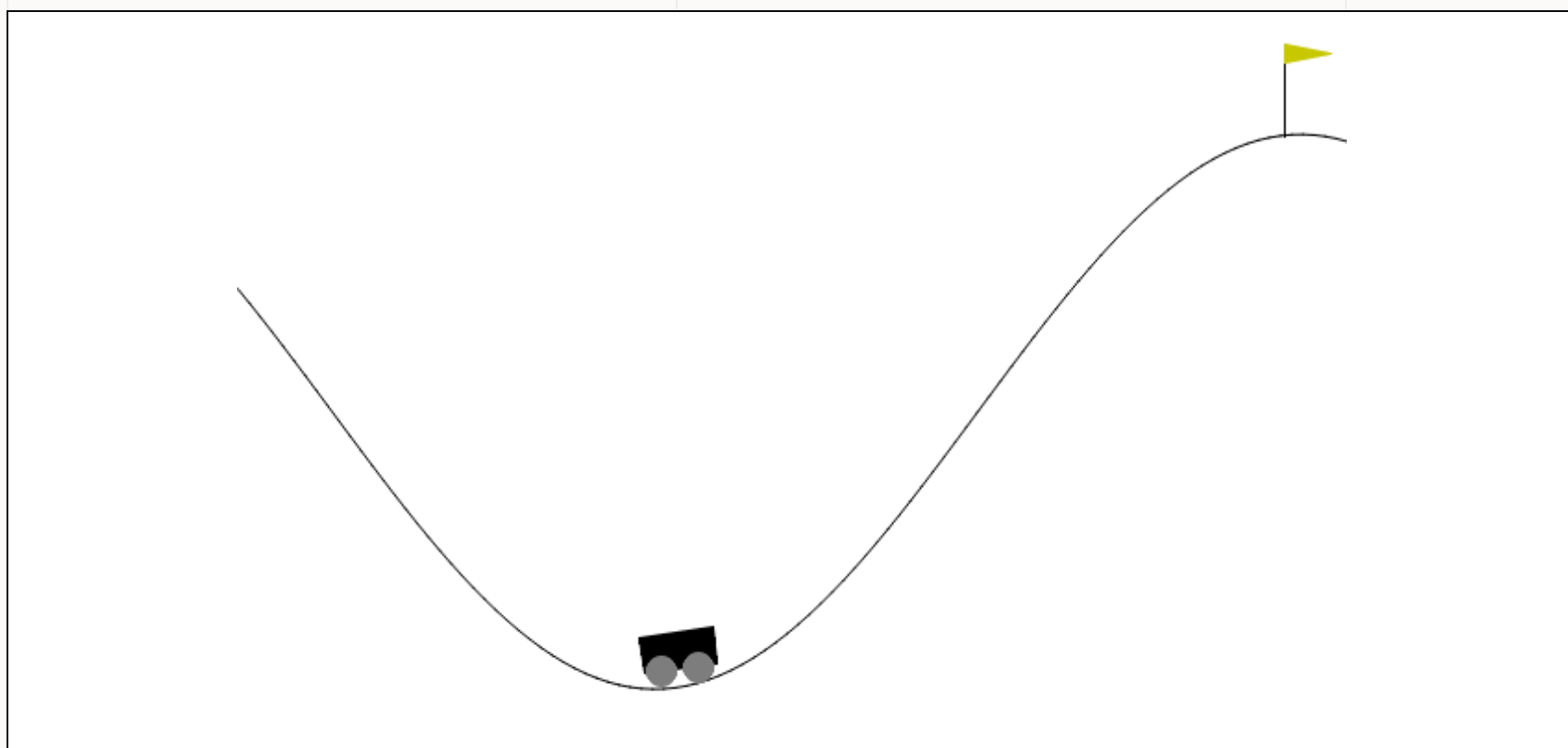
탐험과 이용

MDP 5요소

정책과 가치함수

강화학습이 어려운 이유 I

뒤로 가야 앞으로 갑니다



MountainCar · 학습된 정책이 실제로 움직이는 모습
Farama Gymnasium (MIT License)

01 목표는 오른쪽 언덕 위

그런데 엔진 힘이 **모자랍니다**. 오른쪽으로만 밀면 절대 못 올라가요.

02 그래서 먼저 **왼쪽으로**

목표에서 **멀어지는 손해**를 감수하고 뒤로 물러나 반동을 만듭니다.

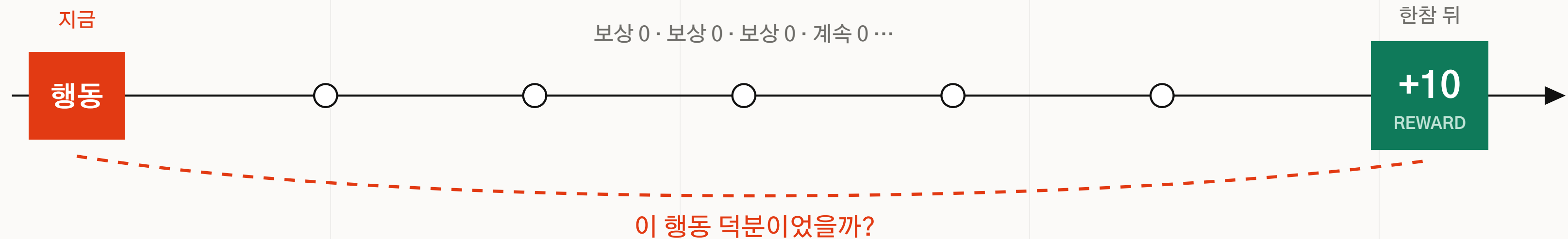
03 그 반동으로 정상까지

지금의 손해가 **미래의 이득**이 됩니다. 이걸 스스로 알아낸 겁니다.

KEY 한 수 한 수를 따로 채점하면 “왼쪽으로 가기”는 **최악의** 수입입니다. 순서 전체를 봐야 정답이 보입니다.

강화학습이 어려운 이유 II

어떤 행동 덕분이었을까?



DELAYED REWARD

지연된 보상

바둑에서 이겼습니다. 그런데 **200수 중 어느 수**가
승리에 기여했을까요?

지도학습이라면

사진을 넣으면 “정답은 고양이”라고 **즉시** 알려줍니다.
채점이 공짜예요.

공부에 비유하면

오늘 두 시간 공부가 **세 달 뒤 시험**에 얼마나
기여했는지, 우리도 모릅니다.

KEY 진짜 난제는 “좋은 행동을 모르는 것”이 아니라 **어떤 행동 덕분이었는지 모르는 것**입니다.

강화학습이 어려운 이유 II

점심, 어디서 드실 건가요?

EXPLOITATION

이용 — 아는 맛집

늘 가던 그 집. 맛은 8점으로 보장됩니다. 실패할 일이 없습니다.

단점 — 근처에 10점짜리 집이 있어도 영영 모릅니다.

EXPLORATION

탐험 — 새로 생긴 집

3점일 수도, 10점일 수도. 결과를 모릅니다.

가치 — 실패해도 “여긴 별로”라는 정보가 남습니다.

둘 다 필요합니다. 탐험만 하면 **매일 도박**이고,
이용만 하면 **영원히 8점**에 머물습니다.

ϵ -greedy: 확률 ϵ 로 아무거나, 나머지는 아는 최선

학습 초반엔 ϵ 를 크게, 갈수록 줄입니다

문제 정의서

MDP
Markov Decision Process

강화학습으로 풀고 싶은 문제가 있다면,
제일 먼저 이 다섯 칸을 채워야 합니다.

$\langle S, A, P, R, \gamma \rangle$

이 튜플 하나가 문제 전체를 정의합니다

예시 — 자율주행이라면?
S: 카메라·라이다 관측 / A: 조향·가감속 / R: 안전·연비·승차감

이 다섯 칸만 채우면 어떤 문제든 강화학습으로 풀 수 있는 형태가 됩니다.

S

상태 집합 — 일어날 수 있는 모든 상황
바둑판의 모든 배치, 로봇의 모든 관절 각도

A

행동 집합 — 고를 수 있는 모든 선택
상하좌우, 액셀·브레이크, 매수·매도

$P(s'|s,a)$

전이 확률 — 원한다고 100% 그대로 되진 않습니다
오른쪽으로 가려 해도 바람에 밀려 아래로 갈 수 있음

$R(s,a)$

보상 함수 — 무엇을 잘한 걸로 칠 것인가
설계자가 정합니다. 가장 어렵고 가장 중요한 부분

γ

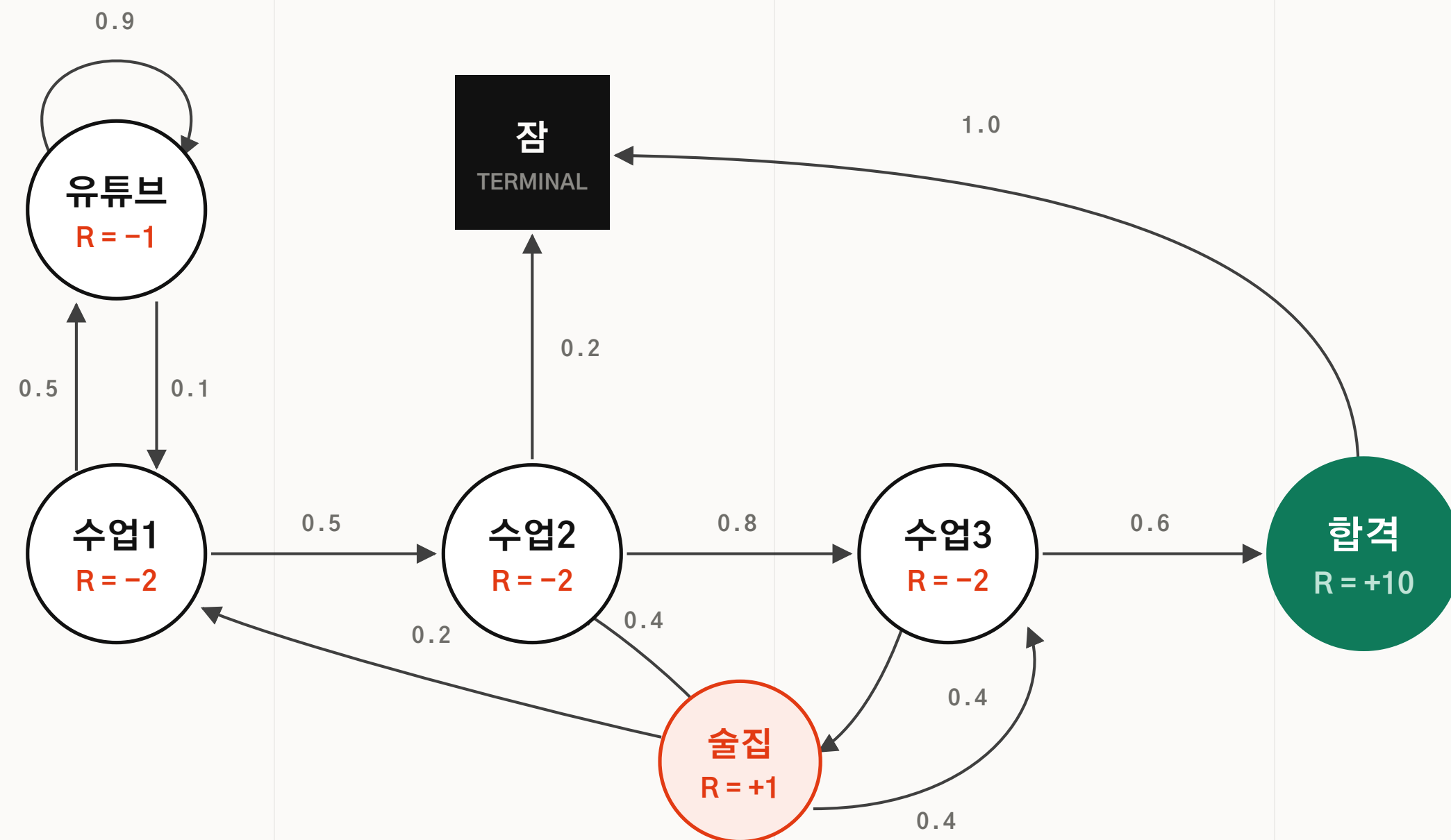
할인율 — 먼 미래를 얼마나 중요하게 볼 것인가
0에 가까우면 근시안, 1에 가까우면 장기 전략가

MDP 읽는 법

어느 대학원생의 하루

- 원 = 상태
- 화살표 = 전이, 위 숫자는 확률
- 붉은 숫자 = 그 상태의 보상

유튜브는 당장 -1로 덜 아프지만, 계속 붙잡히면 +10에 영영 못 갑니다.



MDP의 M

과거는 잊어도 됩니다

$$P(s_{t+1} | s_t) = P(s_{t+1} | s_1, s_2, \dots, s_t)$$

다음 이 어떻게 될지는 “지금”만 알면 된다

현재 상태 **s_t** 가 과거의 모든 정보를 **이미 요약**하고 있다는 가정입니다.

WHY

덕분에 에이전트는 **전체 역사**가 아니라 **현재 한 장면**만 보고 결정할 수 있습니다.

바둑판을 보세요

지금 놓인 돌의 배치만 보면 다음 수를 둘 수 있습니다. **몇 번째 수에 어디를 뒀는지** 순서를 다 외울 필요가 없습니다.

주의 — 정의하기 나름입니다

운전에서 “현재 사진 한 장”은 마르코프가 **아닙니다**. 앞차가 멈춰 있는지 후진 중인지 알 수 없으니까요.

그래서 실무에서는

최근 **4프레임을 겹쳐** 하나의 상태로 씁니다. 속도와 방향까지 담기면 다시 마르코프가 됩니다. — DQN이 실제로 쓴 방법

목표를 정의하기

지금 1개 vs 한 시간 뒤 2개

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

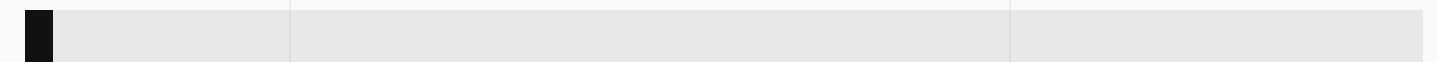
반환값 G — 지금부터 끝까지 받을 보상의 (할인된) 총합

- 강화학습의 목표는 **기대 반환값을 최대화**하는 것. 그게 전부입니다.
- γ 는 미래의 보상을 현재 가치로 환산하는 비율 — 금융의 **이자율**과 같은 개념입니다.
- $\gamma < 1$ 이어야 무한히 이어지는 합이 **발산하지 않습니다**.

10단계 뒤 보상의 현재 가치

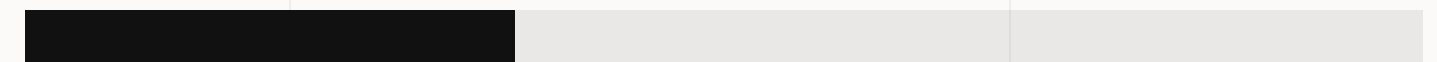
$$\gamma = 0.5$$

근시안 · 0.001



$$\gamma = 0.9$$

균형 · 0.35



$$\gamma = 0.99$$

장기 전략가 · 0.90



γ 를 바꾸면 **에이전트의 성격**이 바뀝니다. 알고리즘이 아니라 성격을요.

우리가 찾는 최종 결과물

정책 π

Policy

“이 상황에서는 이렇게 한다”
— 에이전트의 **처세술**입니다.

앞으로 배울 모든 알고리즘의 **최종 산출물**이 이 π 입니다. 학습이 끝나면 우리 손에 남는 건 π 하나예요.

DETERMINISTIC

결정론적 정책

$$a = \mu(s)$$

“가위바위보는 **무조건 주먹**.” 같은 상황이면 항상 같은 행동. 단순하지만 **읽힙니다**.

STOCHASTIC

확률적 정책

$$\pi(a | s) = P[A_t = a | S_t = s]$$

“주먹 30%, 가위 40%, 보 30%.” 상대가 **읽을 수 없고**, 학습 중에 자연스럽게 **탐험**도 됩니다.

얼마나 좋은 자리인가

상황을 평가할 것인가, 행동을 평가할 것인가

STATE VALUE

$$V^{\pi}(s)$$

상태 가치 — “이 자리, 유리한가?”

체스판을 보고 “지금 백이 유리하네”라고 말하는 것. 앞으로 받을 보상의 기대 총합으로 **상황 자체**를 채점합니다.

한계 — 좋은 자리인 건 알겠는데, 그래서 뭘 두라는 거지?

ACTION VALUE

$$Q^{\pi}(s, a)$$

행동 가치 — “이 수를 두면?”

“여기서 폰을 전진시키면 승률 90%”처럼 **상태 + 행동**을 함께 채점합니다.

강점 — Q만 알면 **가장 높은 값을 고르면 끝**. 바로 행동이 나옵니다.

so 다음 섹션의 **Q-Learning**은 이름 그대로, 이 **Q**를 알아내는 방법에 관한 이야기입니다.

SECTION

03

고전적 강화학습

Bellman Equation · TD Learning · Q-Learning

이제 진짜로 **배웁니다.**

미래를 다 가보지 않고도 학습하는 법.

벨만 방정식

최적 방정식

TD 학습

Q-LEARNING

큰 문제를 작게 쪼개기

벨만 방정식

Bellman Equation

$$V(s) = \mathbb{E} [R + \gamma V(s')]$$

—————
 지금 이 자리의 가치 당장 받는 보상 다음 자리의 가치
 평균적으로 미래를 깎는 할인율

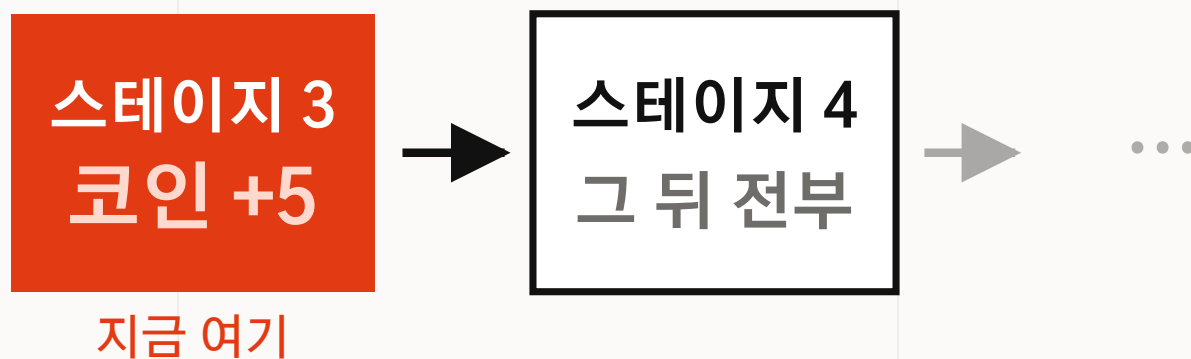
미래 전체를 계산할 필요가 없습니다.

한 걸음 앞만 보면 됩니다.

IDEA

거대한 문제를 “지금 + 나머지”로 쪼개는 것 — 동적 계획법(DP)의 핵심입니다.

비유 · 게임 스테이지



3스테이지의 가치
= 코인 5개 + 4스테이지의 가치

평균 대신 최선

내가 앞으로 실수를 하나도 안 한다면?

BELLMAN EXPECTATION

벨만 기대 방정식

$$V^\pi(s) = \mathbb{E}_\pi[R + \gamma V^\pi(s')]$$

지금 내 습관(π)대로 계속 살면 평균 몇 점일까?

— 현재 실력 진단서



BELLMAN OPTIMALITY

벨만 최적 방정식

$$V^*(s) = \max_a \mathbb{E}[R + \gamma V^*(s')]$$

앞으로 한 번도 실수하지 않는다면 이 자리는 몇 점짜리인가?

— 잠재력 평가서

바뀐 건 딱 하나, $\mathbb{E}$ (평균)이 $\max$ (최선)이 된 것. 이 한 글자가 “최적 정책”을 정의합니다.

손으로 한 번
값은 +1에서부터 먼저 옵니다

S_a	S_b	+1
	S_c	-1

$\gamma = 0.9$ · 이동 보상 0 · 검은 칸은 벽
의도한 방향 80%, 좌우로 새는 확률 각 10%

모든 값을 0에서 시작해도, 골 옆 칸부터 값이 생기고 그 값이 옆으로
번져 나갑니다.

1 단계 · S_b 에서 오른쪽으로

$$V(S_b) = 0 + 0.9(0.8 \times 1 + 0.1 \times 0 + 0.1 \times 0) = 0.72$$

80%는 +1에 도착, 나머지는 아직 값이 0인 칸

2 단계 · S_c 에서 위로

$$V(S_c) = 0.9(0.8 \times 0.72 + 0.1 \times 0 + 0.1 \times (-1)) \approx 0.43$$

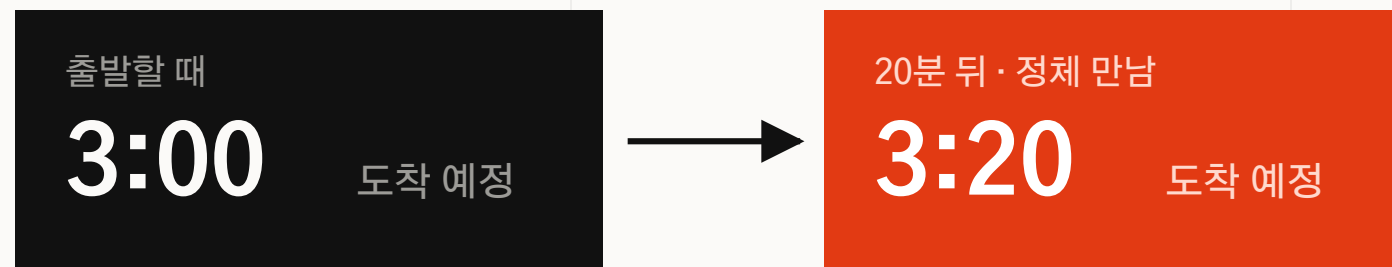
방금 구한 0.72가 재료가 됩니다 — 이게 “재귀”입니다

- 먼 칸일수록 γ 가 여러 번 곱해져 **값이 떨어집니다**. 그래서 골까지의 거리가 자연스럽게 표현됩니다.
- 이 계산을 판 전체에 **반복**하면 값이 수렴합니다 — 가치 반복(Value Iteration).

$\gamma = 0.9$, 이동 보상 0, 의도한 방향으로 갈 확률 80%. 값이 골에서부터 먼저 옵니다.

끝까지 안 가봐도 됩니다

네비게이션은 도착해서 배우지 않습니다



이 +20분이 바로 TD 오차 δ

목적지에 도착할 때까지 기다리지 않습니다

$$V(s) \leftarrow V(s) + \alpha [R + \gamma V(s') - V(s)]$$

이만큼만 반영
원래 믿던 값

새로 알게 된 값

몬테카를로(MC)라면

목적지에 도착한 뒤에야 “아, 3시 20분에 왔네” 하고 배웁니다. 정확하지만 느리고, 끝이 없는 일에는 아예 못 씁니다.

TD

TD 학습이라면

가는 도중에 배웁니다. 아직 도착도 안 했는데 “내 예측이 20분 틀렸네” 하고 즉시 고칩니다.

예측 → 관측 → 오차 → 수정.
이 네 박자가 학습의 전부입니다.

벨만 + TD = ?

거대한 족보 만들기

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

족보의 이 칸을

실제로 받은 보상

원래 믿던 값

학습률

다음 자리 최고점 (추정)

대괄호 안이 **TD 오차** — “실제 + 다음 최고점”과 “내 믿음”의 차이. 그 차이의 **α만큼만** 족보를 고칩니다.

OFF-POLICY 실제로는 **딴짓(탐험)**을 해도, 배울 때는 **정석(max)**으로 배웁니다.

Q - TABLE · 상태 × 행동

상태	↑	↓	←	→
S1	0.31	0.08	0.12	0.72
S2	0.44	0.19	0.05	0.28
S3	0.02	0.61	0.33	0.17
∴ 상태 개수만큼 계속				

각 칸에 “이 상황에서 이 행동의 점수”를 적어둡니다.
실행할 땐 그 줄에서 가장 큰 값을 고르면 끝.

시행착오가 쌓이면

길이 저절로
드러납니다

에이전트는 지도를 받은 적이 없습니다.
수백 번 헤매면서 스스로 지형도를 그렸습니다.

- 처음엔 판 전체가 0
- 골 근처부터 값이 생기고
- 그 값이 옆으로 번져 나갑니다
- 진한 칸을 따라가면 그게 최적 경로

ϵ -greedy로 초반엔 많이 헤매고, 지형도가 그려질수록 헤매는 비율을 줄입니다.

VALUE LANDSCAPE · 색이 진할수록 높은 가치

0.59	0.66	0.73	0.81	0.90	GOAL
0.53	0.59	0.66	0.73	0.81	0.90
0.48	0.53			0.73	0.81
A	0.48	0.53	0.59	0.66	0.73

A = 에이전트 출발점 · 검은 칸 = 벽 · 숫자는 그 칸의 가치 $V(s)$

SECTION

04

딥러닝과의 만남

Deep Q-Network

표가 감당할 수 없을 만큼
세상이 **커졌을 때.**

차원의 저주

함수 근사

EXPERIENCE REPLAY

TARGET NETWORK

왜 딥러닝인가

표가 감당하지 못합니다

문 제 I

상태 공간의 폭발

벽돌깨기 화면 **한 장**의 픽셀 조합만 해도 수십억 가지가 넘습니다. 상태마다 칸 하나씩이면 표를 저장할 **메모리가 지구에 없습니다.**

CURSE OF DIMENSIONALITY

256^{84×84} states

문 제 II

일반화가 안 됩니다

표는 칸마다 **완전히 독립**입니다. 픽셀 하나만 달라져도 **처음 보는 상태**가 되어 참조할 값이 아예 없습니다.

공이 조금 왼쪽에 있을 뿐인데, 방금 배운 걸 하나도 못 씁니다.

해 결

함수 근사

$$Q(s,a) \approx Q(s,a;\theta)$$

표를 **신경망**으로 바꿉니다. 칸을 하나하나 외우는 대신 **규칙(파라미터 θ)**을 배웁니다.

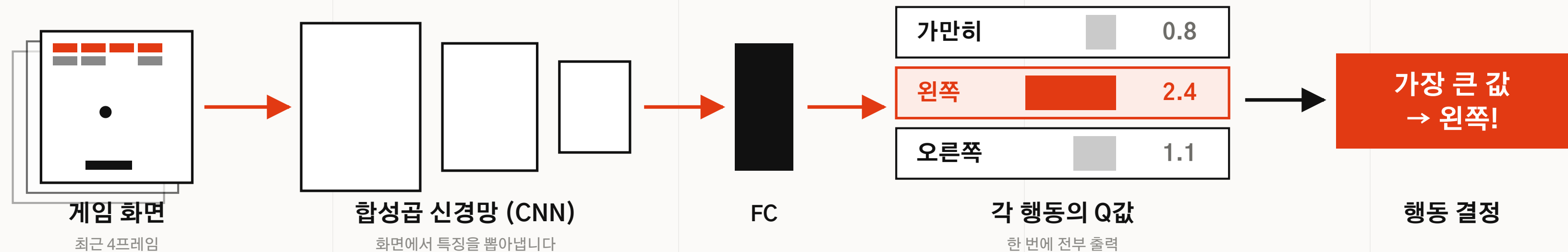
비슷한 상태끼리 학습이 **공유**됩니다 — 처음 보는 화면에도 대응.

SHIFT

암기(표)에서 **이해(함수)**로. 이 한 걸음이 딥 강화학습의 시작입니다.

DeepMind, 2015 · Nature

DQN — 픽셀에서 바로 배우다



사람과 똑같은 입력(화면)만 받고, 게임마다 손대지 않은 **같은 구조**로 49개 Atari 게임을 학습했습니다.

사실 그냥 붙이면 터집니다

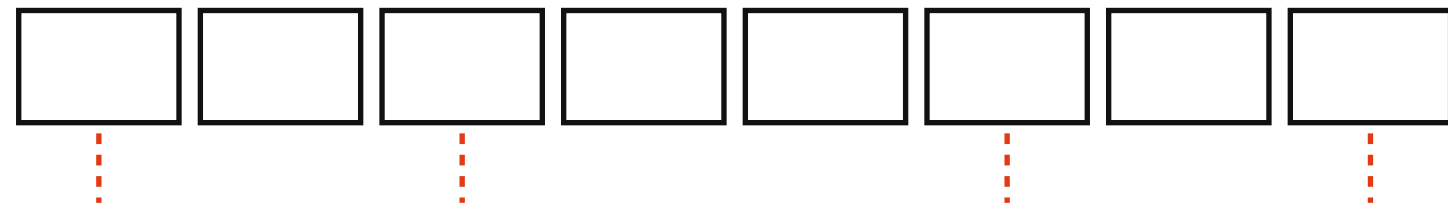
학습을 안정시킨 두 장치

TRICK 1

경험 재생 · Experience Replay

겪은 경험을 **버퍼에 저장**해 두고, 학습할 땐 **무작위로 꺼내** 씁니다.

연속된 경험 — 서로 너무 닮았습니다



무작위로 섞어서 꺼내면 상관관계가 깨집니다



한 번 겪은 경험을 **여러 번 재사용**하는 효과도 덤입니다.

TRICK 2

타깃 네트워크 · Target Network

목표값을 계산하는 네트워크를 **따로 두고 가끔만** 갱신합니다.

목표가 매 스텝 움직이면



잠시 고정해 두면



움직이는 과녁을 향해 쏘면 맞출 수 없습니다. **과녁을 잠깐 세워두는** 장치입니다.

그로부터 10년

DQN은 끝이 아니라 시작이었습니다



NOTE DQN도 벌써 **10년 전** 이야기입니다. 하지만 이 뒤의 모든 것이 — ChatGPT까지 — **방금 배운 그 뼈대** 위에 서 있습니다.

SECTION

05

강화학습의 확장

Physical AI · Large Language Models

게임을 벗어나 **현실의 로봇**과
언어모델까지 온 이야기.

POLICY GRADIENT

ON · OFF-POLICY

PPO · SAC

MCTS · ALPHAZERO

WORLD MODEL

VLA · PHYSICAL AI

RLHF · LLM

Q값을 거치지 않고

정책을 직접 배우기

한 계

로봇 관절 각도는 연속값입니다

“상하좌우 4개” 중 최대를 고르는 건 쉽습니다. 그런데 $-180.0^\circ \sim +180.0^\circ$ 사이
무한한 값 중에서 max를 어떻게 찾죠?

가치 기반 방법이 멈추는 지점입니다.

POLICY GRADIENT

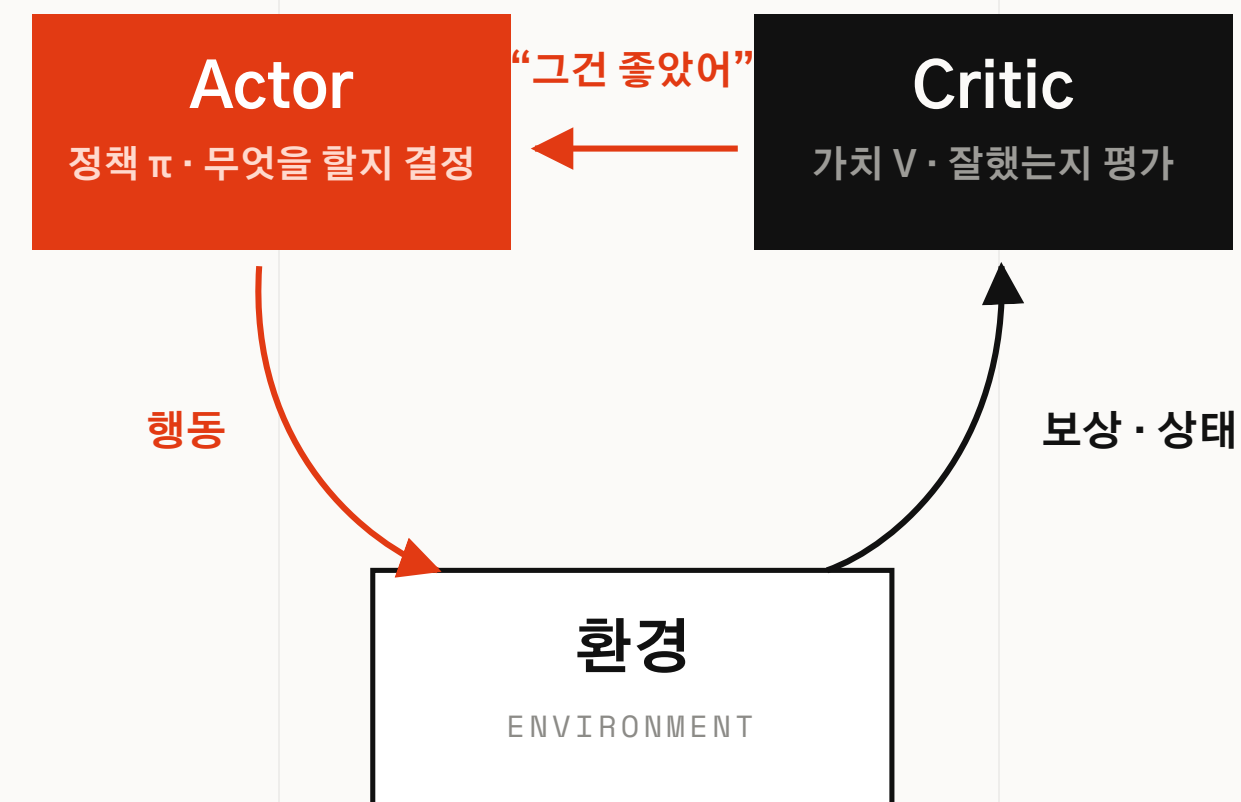
그러면 정책을 직접 고치자

Q값을 계산하지 않고, 정책 $\pi(a|s;\theta)$ 의 파라미터를 **보상이 커지는 방향으로** 직접
밀어줍니다.

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \mathbb{E}[G]$$

보상이 커지는 방향

ACTOR-CRITIC · 역할 분업



Critic의 평가가 Actor의 정책을 갱신합니다 — A2C / A3C 계열

알고리즘을 가르치는 첫 번째 질문

그 경험, 누가 만든 겁니까?

ON-POLICY

직접 몰아본 것만으로 배운다

운전 연습으로 치면 — 내가 핸들을 잡고 겪은 경험만 인정합니다.

정의 행동하는 정책 = 배우는 정책

데이터 정책이 바뀌면 **다시 몰아서** 새로 모아야 함 — 한 번 쓰고 버림

성격 **안정적**이지만 경험을 많이 씁니다 (샘플 비효율)

대표 SARSA · A2C · **PPO**

OFF-POLICY

남의 주행 영상으로도 배운다

블랙박스 영상, 작년 기출 족보 — **남이 만든 경험, 과거의 경험**도 전부 교재가 됩니다.

정의 행동하는 정책 $\neq$ 배우는 정책

데이터 경험을 **리플레이 버퍼**에 쌓아두고 몇 번이고 재사용

성격 **데이터 효율이 좋지만** 학습이 흔들리기 쉬움

대표 Q-Learning · DQN · **SAC**

감각 경험이 **싸면**(게임·시뮬레이터) on-policy로 충분하고, 경험이 **비싸면**(실제 로봇) 한 번의 경험을 우려먹는 off-policy가 절실해집니다.

사실상의 산업 표준 · on-policy

PPO — 욕심내지 말고, 조금씩만

Proximal Policy Optimization · 2017

문제의식

정책을 한 번에 크게 바꾸면 **학습이 그냥 무너집니다**. 어제까지 걸던 로봇이 오늘 못 걸게 되는데, on-policy라 망가진 정책이 만든 데이터로 다시 배워야 하는 악순환.

$$L(\theta) = \mathbb{E}[\min(r\hat{A}, \text{clip}(r, 1 - \varepsilon, 1 + \varepsilon)\hat{A})]$$

새 정책 ÷ 옛 정책

1±ε 밖의 변화는 잘라냄

$\hat{A}$ 는 그 행동이 **평균보다 잘한 정도**. min은 둘 중 **보수적인 쪽**만 믿는다는 뜻 — 아무리 좋아 보여도 1±ε 바깥의 욕심은 내지 않습니다.

비유

골프 레슨의 철칙

코치가 스윙 폼을 **한 번에 다 뜯어고치면** 그날로 폼이 무너집니다. 좋은 코치는 **한 번에 한 뼘씩만** 고칩니다. Proximal — 이름부터 “가까이에서만”입니다.

왜 표준이 됐나

아이디어는 가위질 하나인데 **튜닝에 관대하고 웬만하면 됩니다**. 로봇 보행부터 게임까지 — 그리고 **ChatGPT의 RLHF**에서 다시 만나게 됩니다.

한 줄 애매하면 **PPO** 쓰면 됩니다.

로보틱스의 표준 · off-policy

SAC — 잘하되, 한 길만 파지 마라

Soft Actor-Critic · 2018

$$J(\pi) = \mathbb{E}[\underbrace{\sum R}_{\text{보상은 그대로}} + \underbrace{\alpha}_{\text{둘 사이의 저울 (자동 조절)}} \cdot \underbrace{H(\pi)}_{\text{행동의 다양성 보너스}}]$$

비유

맛집만 다니는 사람의 합정

제일 맛있는 집 **한 곳만** 다니면, 더 맛있는 집을 **영영 못 찾습니다**. SAC은 “안 가본 집에 가는 것 자체”에 보너스를 줘서 *탐험이 끝까지 마르지 않게* 합니다.

덤으로 얻는 것 — 강건함

여러 해법을 고루 익혀두니, 환경이 **조금 달라져도 덜 무너집니다**. 시뮬레이터에서 배워 현실로 가져가는 **Sim-to-Real**에서 특히 빛납니다.

한 걸음 연속 제어 · 실제 로봇이면 **SAC부터**.

$H(\pi)$ = 엔트로피 — 행동이 얼마나 다양한가. “점수를 따라라, 그러나 **새로운 시도에도 월급을 주겠다**”는 목적함수입니다.

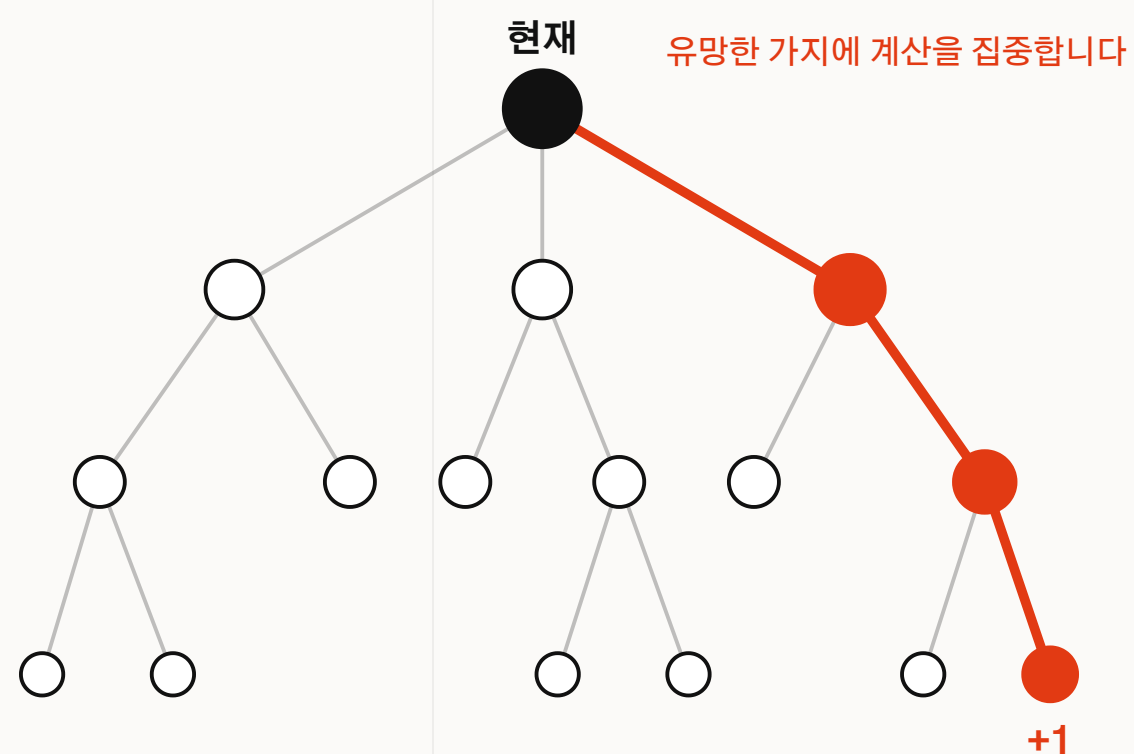
OFF-POLICY의 힘

모든 경험을 **리플레이 버퍼**에 쌓아 재사용합니다. 실제 로봇은 넘어질 때마다 돈이 드니까 — *한 번의 경험을 열 번 우려먹는* 이 성질이 로보틱스에서 결정적입니다.

깊이 생각한다는 것

머릿속에서 미리 두어 보기

Monte Carlo Tree Search



1 Selection · 선택

좋아 보이는 가지를 따라 내려갑니다.

2 Expansion · 확장

새로운 선택지를 하나 열어봅니다.

3 Simulation · 가상 실행

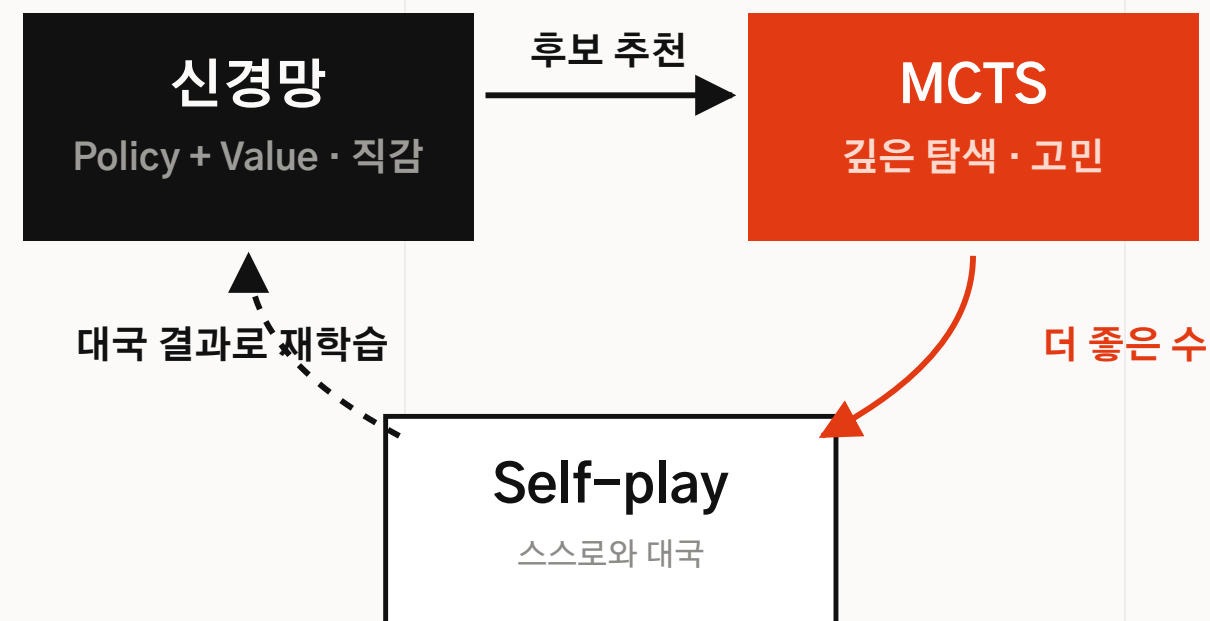
그 뒤를 끝까지 빠르게 **상상으로** 두어 봅니다.

4 Backpropagation · 역전파

결과를 **거꾸로 전달**해 위쪽 점수를 고칩니다.

규칙만 주고 시작

직감과 깊은 고민이 서로를 키웁니다



신경망 = 직감

“여기쯤이 좋아 보인다”를 **즉시** 내놓습니다. 빠르지만 알습니다.

MCTS = 고민

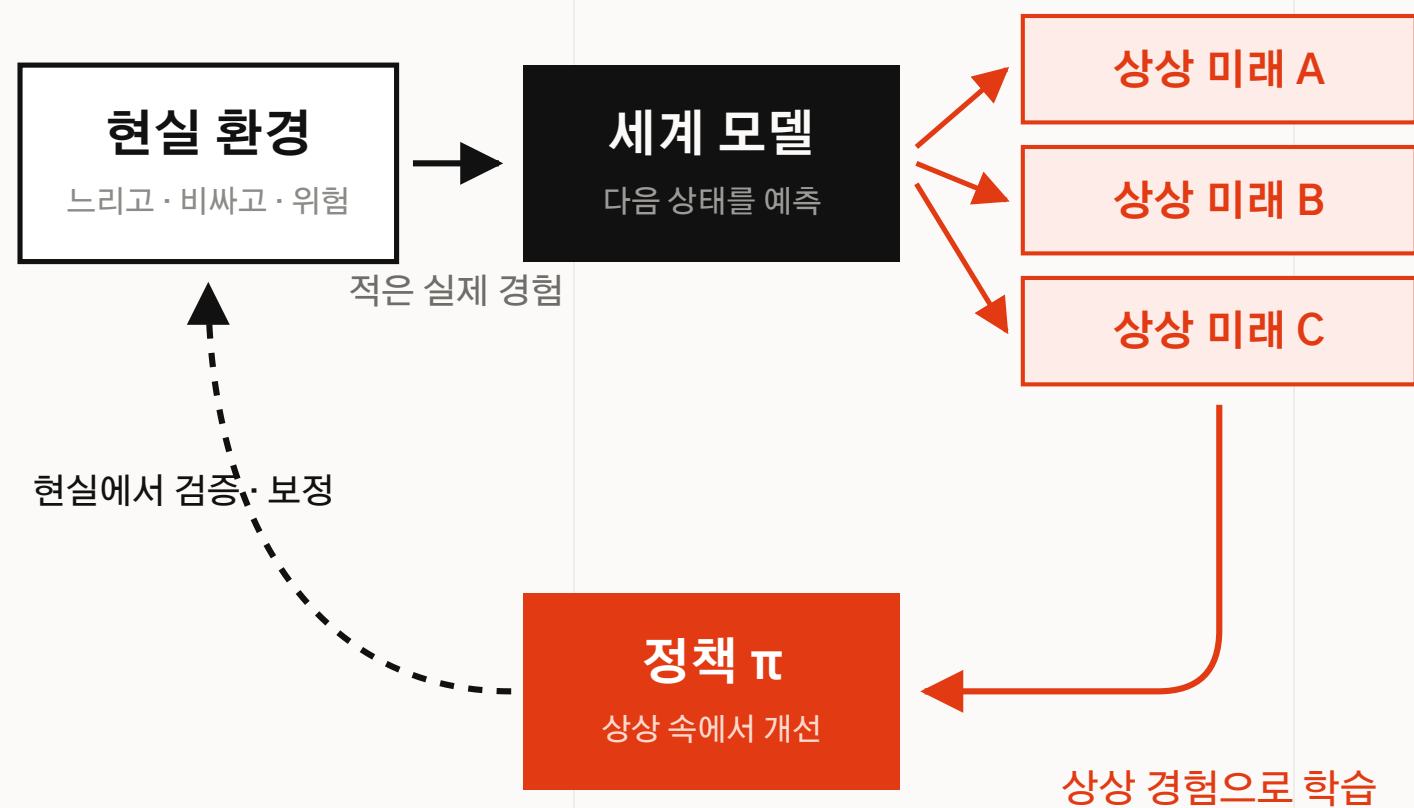
그 후보들의 미래를 **깊게 검증**합니다. 정확하지만 느립니다.

둘을 붙이면

고민해서 찾은 더 좋은 수를 **직감에 다시 가르칩니다**. 그러면 다음 직감이 더 좋아지고, 고민의 출발점도 좋아집니다.

현실은 비쌉니다

상상 속에서 먼저 연습하기



왜 필요한가

로봇을 현실에서 **100만 번 넘어뜨릴** 수는 없습니다. 시간도, 돈도, 로봇도 남아나지 않습니다.

DREAMER

해법

현실 경험은 **조금만** 모아 “세상이 어떻게 움직이는지”를 배우고, 그 다음부터는 **모델 안에서 상상으로** 연습합니다.

OFFLINE RL · 저장된 경험만으로

SAFE RL · 제약을 지키며

MULTI-AGENT RL · 협력과 경쟁

로봇이 말을 알아듣습니다

VLA

Vision · Language · Action

V

본다

카메라 영상에서 물체·공간·상황을 인식합니다.

L

알아듣는다

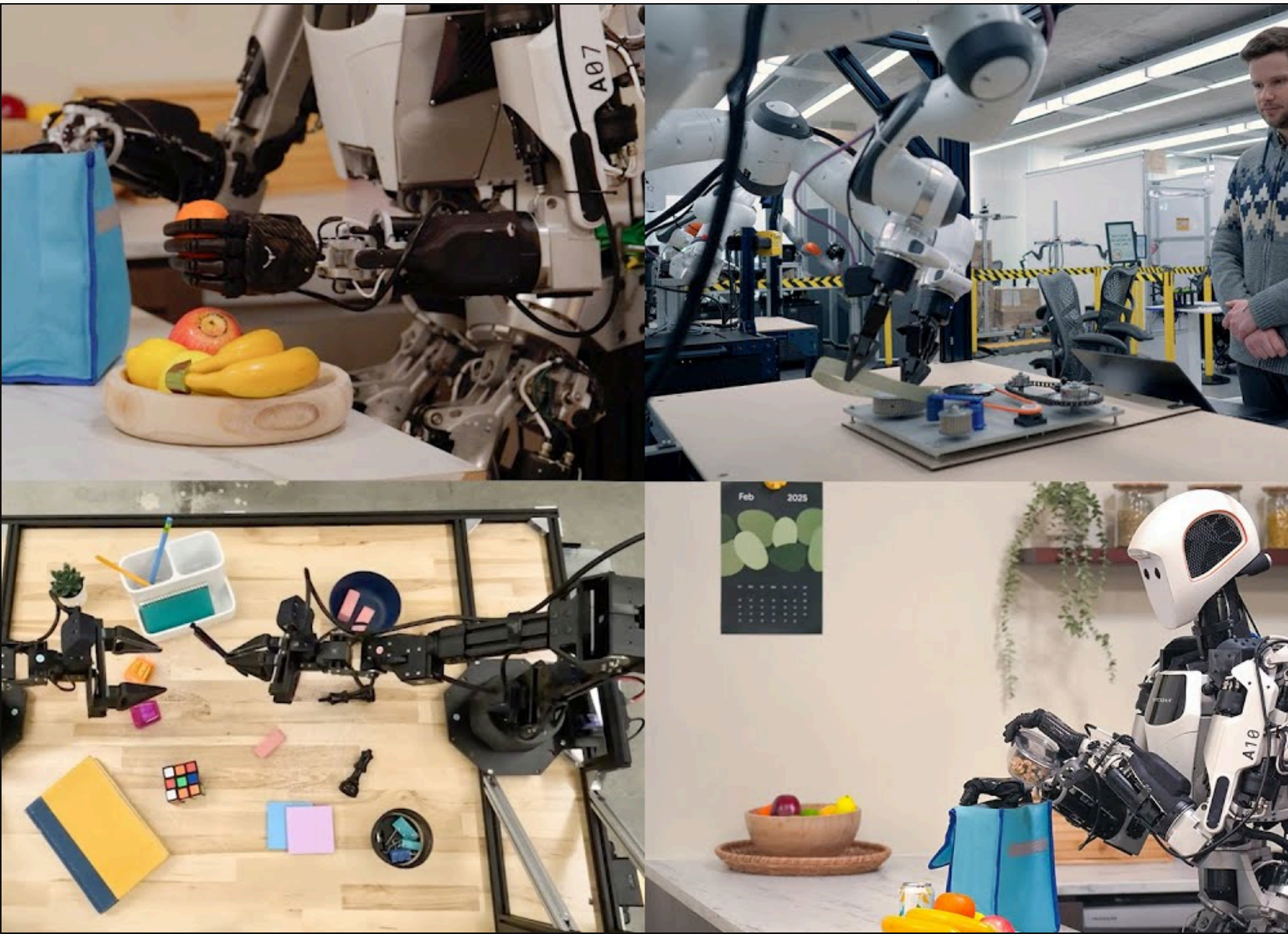
“파란 블록을 상자에 넣어” 같은 자연어 목표를 이해합니다.

A

움직인다

로봇 팔과 관절의 연속적인 움직임을 직접 출력합니다.

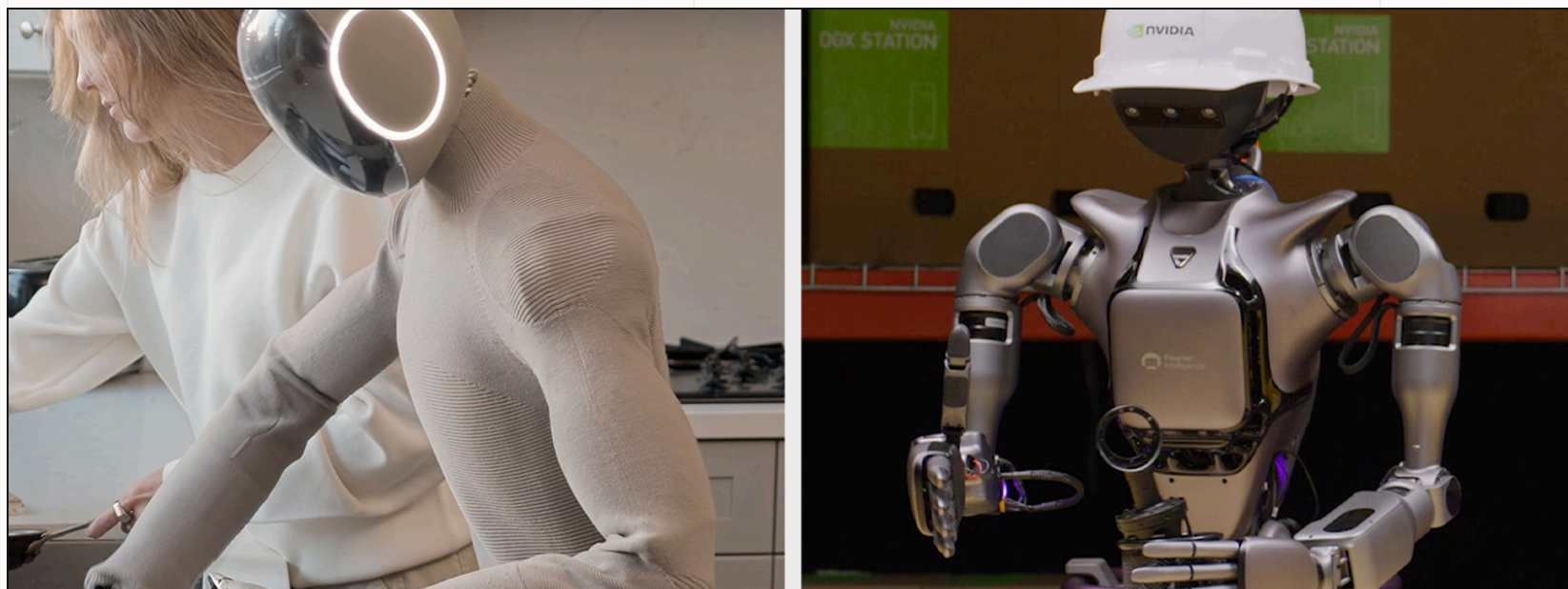
= VLM의 세상 지식 + 로봇 데이터 + 행동 정책



Google DeepMind, Gemini Robotics (2025) · 공식 공개 영상 캡처

현실 데이터는 비싸다

가상에서 연습하고, 현실로 옮긴다



NVIDIA, Isaac GR00T N1 (2025) · 공식 공개 이미지

SIM2REAL

시뮬레이션과 현실의 간극을 좁히는 것이 이 분야의 핵심 과제입니다.

01 Simulation

Digital Twin 안에서 다양한 상황을 **빠르게, 무한히** 만들어냅니다.

02 Data

사람 작업 영상 + 로봇 궤적 + **합성 데이터**를 함께 씁니다.

03 Training

모방학습으로 기본기를, **강화학습**으로 미세 조정을.

04 Sim-to-Real

현실에서 **검증**하고, 실패 경험을 다시 시뮬레이션으로 **되먹입니다**.

ChatGPT를 만든 방법

RLHF — 사람의 선호가 보상이 되다

Reinforcement Learning from Human Feedback

막히는 지점

“좋은 답변”의 보상을
수식으로 쓸 수 있나요?

바둑은 승패가, 게임은 점수가 있습니다. 그런데 **친절하고 정확하고 유용한
답변**에 점수를 매기는 수식은 *세상에 없습니다*.

재 등장 3단계의 “다듬기”에 쓰는 알고리즘이 아까 본
PPO입니다. 오늘 배운 게 그대로 ChatGPT까지
이어집니다.

정 리 **PPO(안정) · SAC(탐험) · RLHF(사람의 선호)** — 셋 다 **Actor-Critic**의 후예입니다. 새 세계가 아니라 같은 뼈대의 개량형이에요.

RLHF · 3단계 파이프라인

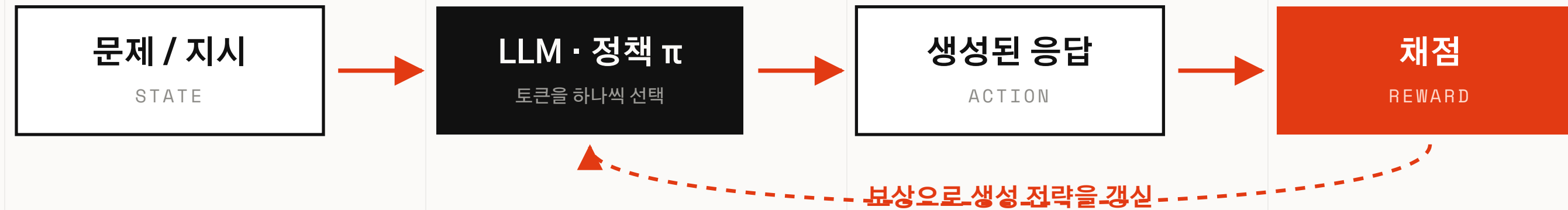
01 사람에게 고르게 한다
답변 A와 B를 보여주고 “어느 쪽이 낫나요?” — 점수 매기기보다 **비교가 훨씬 쉽습니다**.

02 선택을 흉내 내는 채점기를 만든다
그 비교 데이터로 **보상 모델(Reward Model)**을 학습 — 사람 대신 24시간 채점하는
심사위원.

03 그 보상으로 언어모델을 다듬는다
보상 모델을 채점자로 두고 **PPO**로 후학습 — 답변을 생성하고, 채점받고, 조금씩 고칩니다.

언어모델도 에이전트입니다

LLM은 어떻게 “똑똑해지는”가



RLHF

채점자가 **사람**입니다. 답변 두 개 중 나은 것을 고르게 해 보상 모델을 학습합니다.

말투·안전성·도움됨

RLVR

채점자가 **검증기**입니다. 수학은 정답 일치, 코드는 **테스트 통과 여부**로 자동 채점.

사람 손 없이 무한 반복

Agent RL

채점자가 **작업 결과**입니다. 도구를 제대로 썼는지, **일을 끝냈는지**로 보상을 줍니다.

지금 가장 빠르게 움직이는 곳

ONE-PAGE SUMMARY

오늘 배운 것, 한 장으로

01

시행착오로 배운다

정답 대신 **보상**. 결과 신호만으로 행동을 고쳐 나갑니다.

02

MDP로 적는다

$\langle S, A, P, R, \gamma \rangle$. 문제를 이 다섯 칸으로 옮기면 풀 수 있는 형태가 됩니다.

03

한 걸음 앞만 본다

벨만 + TD. 끝까지 안 가보고 **예측**을 계속 고칩니다.

04

표에서 신경망으로

암기 대신 **일반화**. DQN이 픽셀에서 직접 배우는 길을 열었습니다.

05 · 지금 최전선

로봇은 상상 속에서 연습하고, 언어모델은 결과로 후학습합니다. 40년 된 아이디어 하나가 지금 가장 앞선 두 분야를 동시에 밀고 있습니다.

기억할 문장은 하나입니다.

“예측하고, 겪어보고, 틀린 만큼 고친다.”

WHAT'S NEXT

더 공부하고 싶다면

순서대로 하실 필요는 없습니다.

3번을 먼저 권합니다 — 직접 돌려보는
게 제일 빠릅니다.

- | | |
|-----------|---|
| 01 | Sutton & Barto , <i>Reinforcement Learning: An Introduction</i>
이 분야의 교과서. 무료로 공개되어 있습니다. |
| 02 | David Silver , UCL RL Course
알파고를 만든 사람의 강의. 오늘 내용의 확장판입니다. |
| 03 | Gymnasium + Stable-Baselines3
오늘 본 CartPole·FrozenLake가 전부 여기 있습니다. 20줄이면 돌아갑니다. |
| 04 | Spinning Up in Deep RL (OpenAI)
이론과 구현을 함께 보여주는 실전 자료. |

Q&A

무엇이든 물어보세요.

윤서진

AI Researcher · HD한국조선해양 (KSOE)

서강대학교 AI · SW대학원 웨비나